

DEVOPS

Let's
Build
Together 😊

1. DEFINITION

DevOps is a combination of "Development" and "Operations".

It is a culture, practice and set of tools that improves an organization's ability to deliver applications and services at high velocity.



Goal → Deliver Value
Faster, Better, Cheaper

2. INTRODUCTION

- Traditional approach had gap between Dev and Ops.
- DevOps breaks these silos.
- Focus on automation, collaboration and continuous improvement.
- Helps in rapid delivery of high-quality software.

3. WHY DEVOPS ?

- ✓ Faster Delivery 🚀
- ✓ Improved Collaboration 👤
- ✓ Better Quality 🛡️
- ✓ Continuous Improvement 📈
- ✓ Efficient Use of Resources ⚙️
- ✓ Customer Satisfaction 😊

REAL-WORLD UNDERSTANDING

EX: Think of a Food Delivery App

- Dev builds new features
- Ops makes sure app is always available to users
- DevOps makes sure new features are released quickly without breaking the app.

= Happy Customer 😊

KEY POINTS

- DevOps is not a tool, it's a Mindset.
- People + Process + Tools = DevOps Success
- Automation is the ❤️ of DevOps.
- It's an ongoing journey, not a destination.

Quick Notes

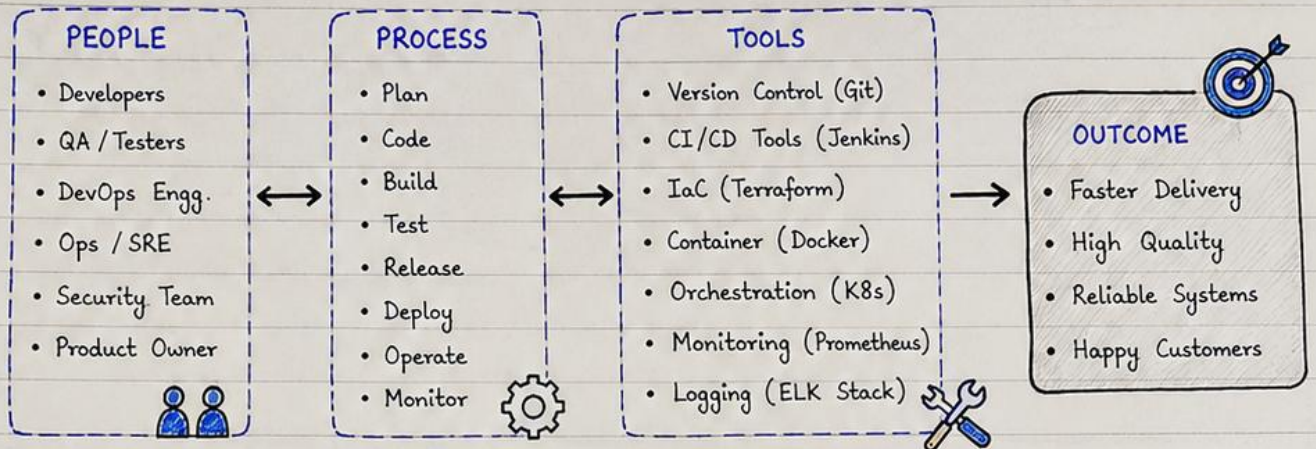
DevOps → Faster Delivery
Better Collaboration
Continuous Improvement

“ Continuous Delivery is the foundation, DevOps is the Culture. ”

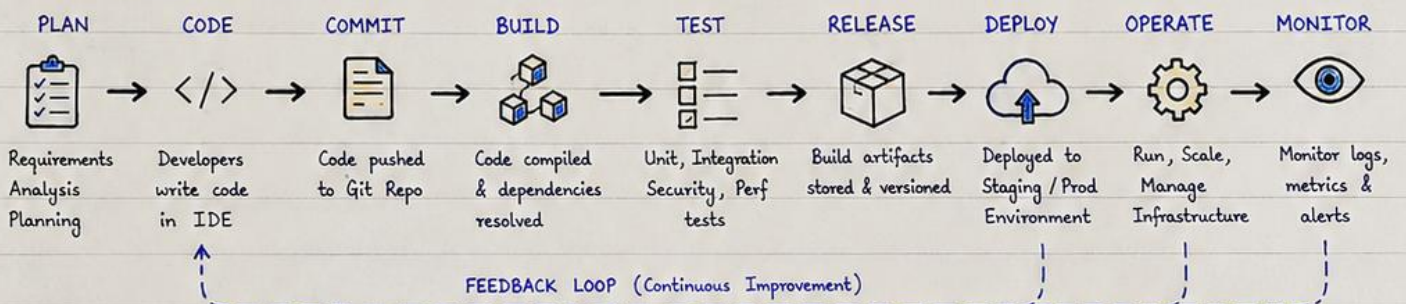
2. DEVOPS ARCHITECTURE

DevOps architecture connects **People**, **Process** and **Tools** to deliver value continuously.

Page
2 of 8



3. DEVOPS WORKING (CONTINUOUS FLOW)



4. KEY COMPONENTS

- ★ **CI (Continuous Integration)** - Code integrated frequently into shared repo.
- ★ **CD (Continuous Delivery)** - Automated delivery of code to staging environment.
- ★ **IaC (Infrastructure as Code)** - Manage Infra using code (Terraform, Ansible).
- ★ **Monitoring** - Real-time visibility of system health & performance.
- ★ **Logging** - Centralized logs for troubleshooting.
- ★ **Security** - Integrated at every stage (DevSecOps).

ARCHITECTURE LAYERS

	User Layer	Users access app
	Application Layer	Your application
	Automation Layer	CI/CD, IaC, Scripts
	Infrastructure Layer	Servers, Cloud, Network
	Monitoring Layer	Logs, Metrics, Alerts

REAL-WORLD EXAMPLE

Think of Amazon → Code commit → Build → Test → Deploy → Monitor 24/7 across thousands of servers globally. This is DevOps in action!

Quick Notes

DevOps is not just about tools, it's about culture, automation and continuous improvement.



DEVOPS LIFECYCLE

Page
3 of 8

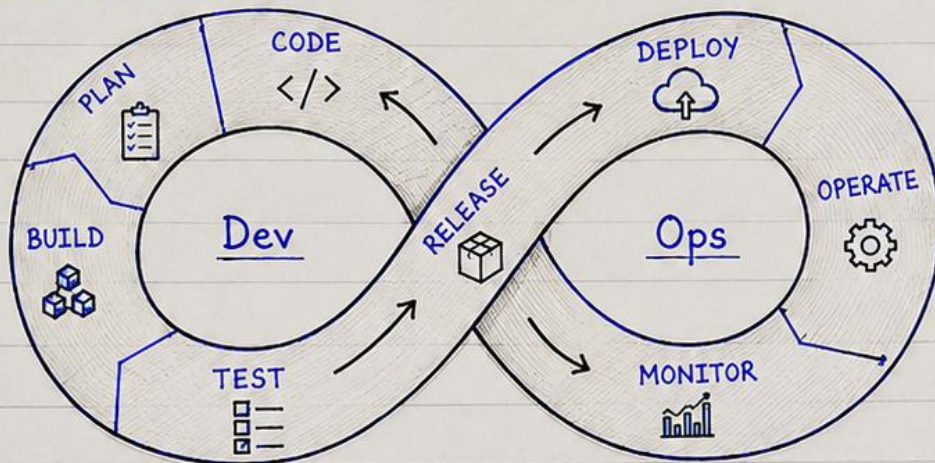
DEFINITION :

DevOps Lifecycle is a continuous process that helps deliver high-quality software faster, more reliably and with continuous improvement.

KEY POINTS

- End-to-End Process
- Continuous Flow
- Collaboration
- Automation
- Feedback Driven

THE INFINITY LOOP



REAL-WORLD UNDERSTANDING

Think of it like a **FOOD ORDER** from planning the menu to delivering it hot to the customer and keeps improving based on feedback.

LIFECYCLE STAGES (1-LINE EXPLANATION)

	1. PLAN	Define <u>requirements</u> , scope, timeline and resources.
	2. CODE	Developers <u>write</u> and commit code into version control system.
	3. BUILD	Code is compiled and dependencies are resolved to create <u>build artifact</u> .
	4. TEST	Automated tests are executed to ensure <u>code quality</u> and functionality.
	5. RELEASE	Approved build is packaged and <u>versioned</u> for deployment.
	6. DEPLOY	Application is deployed to <u>target environment</u> (Staging/Prod).
	7. OPERATE	Application is run, managed and users are served. Ensure <u>reliability</u> and <u>performance</u> .
	8. MONITOR	Monitor system health, lags, metrics and user behavior. Collect <u>feedback</u> for continuous improvement.

QUICK NOTES

DevOps Lifecycle never stops. It's a continuous loop of build, measure and improve.

REMEMBER

- ✓ Automation + Collaboration = Speed ⚡
- ✓ Feedback + Monitoring = Quality ★
- ✓ Dev + Ops Culture = Success 🏆

MOTTO:

Build → Test → Release → Deploy → Operate → Monitor → Repeat ∞

DEVOPS

Benefits &
Real-World
Example

BENEFITS OF DEVOPS



1. **FASTER DELIVERY** → Code changes reach users quickly through automated pipeline.



2. **BETTER COLLABORATION** → Dev, Ops, QA, Security work together. No more silos.



3. **AUTOMATION** → Reduces manual work, human errors and increases consistency.



4. **CONTINUOUS FEEDBACK** → Get real-time feedback from users, logs, metrics to improve continuously.



5. **SCALABILITY** → Applications can scale up/down based on demand automatically.



6. **RELIABILITY** → Automated tests, monitoring and rollback ensure stable systems.

WHY IMPORTANT ?

In today's world,
Speed = Competitive
Advantage



QUICK NOTES

- ✓ Happy Team 😊
- ✓ Happy Customer 😊
- ✓ Lower Cost ↓\$
- ✓ Better Quality ★

REAL-WORLD EXAMPLE (END-TO-END FLOW)

1. Developer
Code Push



Code pushed to
Git Repository

2. Jenkins
Build



Jenkins pulls code,
builds and runs
tests

3. Docker
Image



Docker image
is created

4. Kubernetes
Deploy



K8s pulls image
and deploys
the container

5. Prometheus
Monitor



Prometheus collects
metrics and monitors
health

Users
Access App



Users access the
application ❤️

FLOW SUMMARY

Developer code push karta hai → Jenkins build karta hai →
Docker image banti hai → Kubernetes deploy karta hai →
Prometheus monitor karta hai.

RESULT

- ✓ Fast Delivery
- ✓ High Quality
- ✓ Happy Users

KEY TAKEAWAY

DevOps is not just about tools, it's about mindset, process
and people working together to deliver value continuously. ★

“ Automate Everything
Measure Everything
Improve Continuously ”

DEVOPS TOOLS

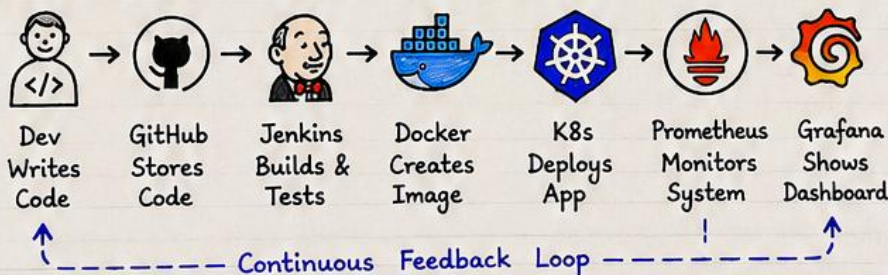
★
Right Tool
at Right Time
= DevOps Success

DevOps uses different tools to Automate, Build, Deploy, Monitor and Improve continuously.



TOOL	PURPOSE (1-LINE)	CATEGORY
①  Git	→ Distributed <u>Version Control</u> System to track changes in code.	 Version Control
②  GitHub / GitLab	→ Host code online, <u>Collaboration</u> PR, Issues, CI/CD integration.	 Code Hosting & Collaboration
③  Jenkins	→ Automate Build, Test and Deploy using <u>Pipelines</u> .	 CI/CD Tool
④  Docker	→ Create lightweight <u>Containers</u> to package and run applications.	 Containerization
⑤  Kubernetes (K8s)	→ <u>Orchestrate</u> Containers, Auto-scaling, Self-healing, Load balancing.	 Container Orchestration
⑥  Terraform	→ <u>Infrastructure as Code (IaC)</u> - Provision Cloud Resources automatically.	 IaC / Cloud Provisioning
⑦  Ansible	→ Configuration Management & <u>Automation</u> - Manage servers easily.	 Automation & Configuration
⑧  Prometheus	→ Monitor Systems, Collect <u>Metrics</u> , Generate Alerts.	 Monitoring & Alerting
⑨  Grafana	→ Visualize Metrics from Prometheus in Beautiful <u>Dashboards</u> .	 Visualization & Dashboard

HOW THESE TOOLS WORK TOGETHER ?



QUICK NOTES

- ✓ No Single Tool is Best
- ✓ Combination of Tools Makes DevOps Powerful.
- ✓ Choose Tools based on Project Needs.
- ✓ Learn Basics First, Then Go Deep.

★ KEY TAKEAWAYS

- ✓ Tools are enablers, not the goal.
- ✓ Automation + Right Tools = Faster Delivery.
- ✓ Monitor Everything, Improve Continuously.
- ✓ Keep it Simple, Scalable and Reliable.

REMEMBER

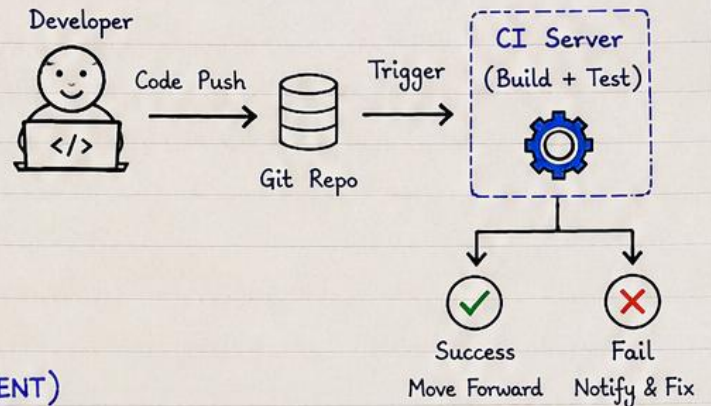
Tools change with time,
But Concepts remain same.

CI / CD in DEVOPS

Automate
Everything
Deliver
Continuously 😊

1. CI (CONTINUOUS INTEGRATION)

- Developers integrate code into shared repository frequently.
- Every integration triggers an automated build and test.
- Goal is to detect issues early and keep codebase stable.



2. CD (CONTINUOUS DELIVERY / DEPLOYMENT)

- After successful testing, code is automatically prepared for release.
- Continuous Delivery → Manual approval before deploy.
- Continuous Deployment → Auto deploy to production.
- Goal is to deliver value to users faster and reliably.

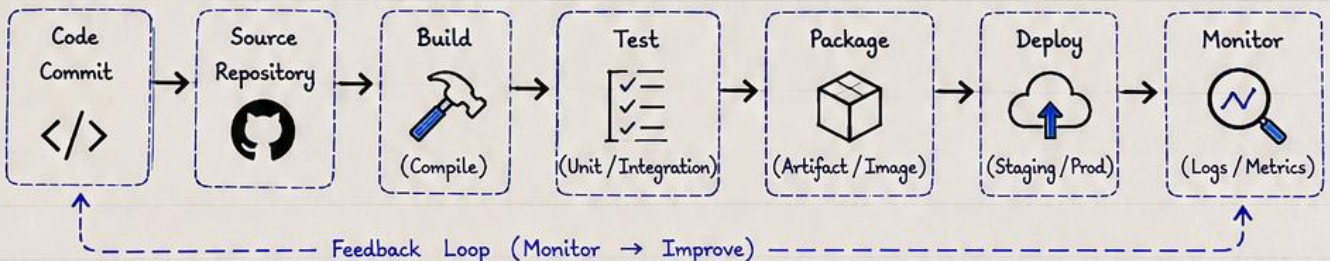
CONTINUOUS DELIVERY (CD)

Code is always in deployable state.
Human approval before production. 👤

CONTINUOUS DEPLOYMENT (CD)

Code is automatically deployed to production after all tests pass. 🤖

3. SIMPLE CI/CD PIPELINE DIAGRAM



4. SHORT WORKFLOW

- Developer writes code and commits to Git.
- Code pushed to GitHub / GitLab.
- CI server (e.g., Jenkins) pulls code.
- Build is triggered and code is compiled.
- Automated tests run (unit + integration).
- If tests pass → Artifact / Docker image is created.
- CD process deploys to staging / production.
- Monitoring tools check health and performance.
- Feedback collected and cycle repeats.

Automate
→ Build
→ Test
→ Deploy
→ Monitor
→ Repeat



QUICK NOTES

- ✓ CI = Integrate Early, Test Early
- ✓ CD = Deliver Fast, Deploy Often
- ✓ Automation reduces human errors.
- ✓ Small changes, frequent releases = Happy Users 😊
- ✓ Always monitor and improve. ★

★ INTERVIEW TIP

CI focuses on "Integration + Testing"
CD focuses on "Delivery + Deployment"



Together they form the backbone of DevOps.

DEVOPS INTERVIEW QUESTIONS

Prepare Well
Crack Any
Interview! 📌

1. Q. What is DevOps?

Ans: DevOps is a culture and set of practices that connects Dev (Development) and Ops (Operations) to deliver software faster and reliably.

2. Q. Why DevOps?

Ans: To deliver software faster, improve quality, increase collaboration and automate processes.

3. Q. Difference between Agile & DevOps?

Ans: Agile is about iterative development.
DevOps is about automating delivery and operations + culture.

Agile	DevOps
• Focus on Dev	• End-to-End
• Iterative	• Continuous
• Team Centric	• Culture + Tools

4. Q. What is CI?

Ans: Continuous Integration → Developers integrate code frequently, automated build & test run.

5. Q. What is CD?

Ans: Continuous Delivery/Deployment → Code is always ready to release / auto deployed to production.

6. Q. What is the difference between CI&CD?

Ans: CI = Integrate & Test
CD = Deliver & Deploy

7. Q. What is Infrastructure as Code (IaC)?

Ans: Managing infrastructure using code (Terraform, CloudFormation, Pulumi, etc.).

8. Q. What is Automation in DevOps?

Ans: Using tools/scripts to perform tasks automatically without manual intervention.

9. Q. What is a Pipeline?

Ans: A series of automated steps to build, test, package and deploy the application.

10. Q. What is Version Control System?

Ans: System that records changes to files over time (e.g., Git).

11. Q. What is a Repository?

Ans: Storage location in VCS where code is stored (e.g., GitHub repo).

12. Q. What is Docker?

Ans: Platform to containerize applications so they run consistently everywhere.

13. Q. What is Kubernetes?

Ans: Container orchestration tool to deploy, scale and manage containers.

14. Q. What is configuration management?

Ans: Managing system configurations using tools like Ansible, Puppet, Chef.

15. Q. What is Monitoring?

Ans: Tracking system metrics, logs and alerts to ensure performance and availability.

16. Q. What is the role of Jenkins in DevOps?

Ans: Jenkins is a CI/CD tool used to automate build, test and deployment.

17. Q. What is the purpose of Ansible?

Ans: Configuration management and application deployment tool using simple YAML playbooks.

18. Q. What is meant by "Shift Left"?

Ans: Testing and security moved early in the SDLC to find issues before production.

19. Q. What is a Container?

Ans: Lightweight, standalone package with app + dependencies.

20. Q. What is Immutable Infrastructure?

Ans: Infrastructure that is never modified after deployment; replace instead of change.

QUICK TIP

CI → Build & Test
CD → Release
CDE → Deploy

IMPORTANT POINTS

- ✓ Culture > Tools
- ✓ Automate Everything
- ✓ Measure Everything
- ✓ Collaborate Always
- ✓ Learn Continuously

REMEMBER

- People
- Process
- Tools
- Automation
- Measurement
- = Successful DevOps

COMMON INTERVIEW TOPICS

- ★ Linux Basics
- ★ Networking Basics
- ★ Cloud (AWS/Azure)
- ★ Git & GitHub
- ★ CI/CD Tools
- ★ Containers & K8s
- ★ IaC (Terraform)
- ★ Monitoring Tools
- ★ Scripting (Bash)

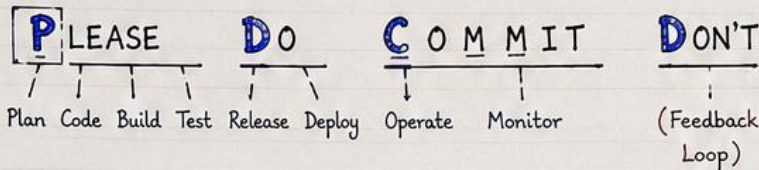


DEVOPS

CHEAT SHEET + MEMORY TRICKS

One Page
Revision

DEVOPS LIFECYCLE (MNEMONIC)

Or Simply Remember → **P C B T R D O M**

Plan → Code → Build → Test → Release → Deploy

→ Operate → Monitor (Continuous Feedback)

IMPORTANT KEYWORDS

- Automation
- Collaboration
- Continuous Integration (CI)
- Continuous Delivery (CD)
- Infrastructure as Code (IaC)
- Version Control
- Containerization
- Orchestration
- Monitoring
- Feedback Loop
- Scalability
- Reliability
- Security Shift-Left
- Idempotent
- Immutable Infrastructure

DEVOPS TOOLS QUICK MAP

- Git** → Version Control System
- GitHub/GitLab** → Code Hosting & Collaboration
- Jenkins** → CI/CD Automation Server
- Docker** → Containerization Platform
- Kubernetes** → Container Orchestration
- Terraform** → Infrastructure as Code
- Ansible** → Configuration Automation
- Prometheus** → Monitoring & Alerting
- Grafana** → Visualization & Dashboards

QUICK INTERVIEW TIPS

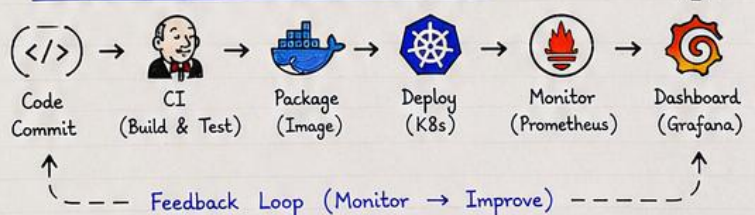
- ✓ Understand concepts before tools.
- ✓ Give real-world examples.
- ✓ Keep answers short, clear & structured.
- ✓ Highlight automation, monitoring and collaboration.
- ✓ Always end with how it adds value to business.
- ✓ Be honest, confident & calm 😊

DEVOPS PRINCIPLES (GOLDEN RULES) ☆

- ✓ Automate Everything Possible.
- ✓ Build Once, Deploy Anywhere.
- ✓ Monitor Everything, Improve Continuously.
- ✓ Fail Fast, Learn Fast.
- ✓ Collaboration over Communication.
- ✓ Security is Everyone's Responsibility.
- ✓ Keep It Simple, Scalable & Repeatable.



SHORT WORKFLOW (REVISION)



RAPID FIRE REVISION

- DevOps = Culture + Automation + Measurement
- CI = Integrate & Test Early
- CD = Deliver Fast & Deploy Often
- IaC = Manage Infra using Code
- Containers = Consistent Environment
- K8s = Orchestrate Containers at Scale
- Monitoring = Visibility
- Feedback Loop = Continuous Improvement
- Version Control = Track & Manage Changes
- Automation = Save Time, Reduce Errors
- Collaboration = High Performance Teams

Remember

Small changes done continuously make a **BIG** impact. 😊

MANTRA

★ Automate ★ Collaborate ★ Monitor ★ Improve ★ Deliver Value



Follow for More DevOps Notes 🚀

LinkedIn: @dileep kumar mishra

Instagram: @dileep.devops



Keep Learning & Growing ❤️